

1. [2 pts] Environment Setup Documentation:

- o Screenshots or logs showing successful execution of Docker Compose, package installation, and server startup.
- o Documentation of any errors encountered during setup and the steps you took to resolve them.

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26100.3624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\junwe>cd Downloads\feng12

C:\Users\junwe\Downloads\feng12>docker-compose up
no configuration file provided: not found

C:\Users\junwe\Downloads\feng12>dir
Volume in drive C is OS
Volume Serial Number is F67F-4E3F

Directory of C:\Users\junwe\Downloads\feng12

2025/04/07 01:41 <DIR> .
2025/04/07 01:41 <DIR> ..
2025/04/07 01:41 <DIR> server
0 File(s) 0 bytes
3 Dir(s) 34,020,810,752 bytes free

C:\Users\junwe\Downloads\feng12>cd server

C:\Users\junwe\Downloads\feng12\server>dir
Volume in drive C is OS
Volume Serial Number is F67F-4E3F

Directory of C:\Users\junwe\Downloads\feng12\server

2025/04/07 01:41 <DIR> .
2025/04/07 01:41 <DIR> ..
2025/04/07 01:41 572 docker-compose.yaml
2025/04/07 01:41 10,390 server.py
2 File(s) 10,962 bytes
2 Dir(s) 34,020,294,656 bytes free

C:\Users\junwe\Downloads\feng12\server>docker-compose up
time="2025-04-07T01:44:11-04:00" level=warning msg="C:\Users\junwe\Downloads\feng12\server\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Running 3/3
  >db Pulled 25.7s
  >phpmyadmin Pulled 22.4s
```

```
C:\WINDOWS\system32\cmd. X + v

[*] Running 3/3
  >Network server_default Created 0.1s
  >Container db Created 0.5s
  >Container server-phpmyadmin-1 Created 0.1s
Attaching to db, phpmyadmin-1
db | 2025-04-07 05:44:39+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 9.2.0-1.el9 started.
db | 2025-04-07 05:44:39+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
db | 2025-04-07 05:44:39+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 9.2.0-1.el9 started.
phpmyadmin-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.18.0.3. Set the 'ServerName' directive globally to suppress this message
phpmyadmin-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.18.0.3. Set the 'ServerName' directive globally to suppress this message
phpmyadmin-1 | [Mon Apr 07 05:44:39.828024 2025] [mpm_prefork:notice] [pid 1:tid 1] AH00163: Apache/2.4.62 (Debian) PHP/8.2.27 configured -- resuming normal operations
phpmyadmin-1 | [Mon Apr 07 05:44:39.828085 2025] [core:notice] [pid 1:tid 1] AH00094: Command Line: 'apache2 -D FOREGROUND'
db | 2025-04-07 05:44:39+00:00 [Note] [Entrypoint]: Initializing database files
db | 2025-04-07 05:44:40.011922 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
db | 2025-04-07 05:44:40.012657 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 9.2.0) initializing of server in progress as process 82
db | 2025-04-07 05:44:40.021732 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
db | 2025-04-07 05:44:40.320437 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
db | 2025-04-07 05:44:41.402902 6 [Warning] [MY-010403] [Server] root@localhost is created with an empty password ! Please consider switching off the --initialize-insecure option.
db | 2025-04-07 05:44:43.128161 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
db | 2025-04-07 05:44:43+00:00 [Note] [Entrypoint]: Database files initialized
db | 2025-04-07 05:44:43+00:00 [Note] [Entrypoint]: Starting temporary server
db | 2025-04-07 05:44:43.246678 0 [System] [MY-015015] [Server] MySQL Server - start.
db | 2025-04-07 05:44:43.459621 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 9.2.0) starting as process 127
db | 2025-04-07 05:44:43.478528 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
db | 2025-04-07 05:44:43.802882 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
db | 2025-04-07 05:44:44.151252 0 [Warning] [MY-010060] [Server] CA certificate ca.pem is self signed.
db | 2025-04-07 05:44:44.151252 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
db | 2025-04-07 05:44:44.154525 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
db | 2025-04-07 05:44:44.183587 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqlx.sock
db | 2025-04-07 05:44:44.183682 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '9.2.0' socket: '/var/run/mysqld/mysqld.sock' port: 0 MySQL Community Server - GPL.
db | 2025-04-07 05:44:44+00:00 [Note] [Entrypoint]: Temporary server started.
db | /var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
db | Warning: Unable to load '/usr/share/zoneinfo/iso3166.tab' as time zone. Skipping it.
db | Warning: Unable to load '/usr/share/zoneinfo/leap-seconds.list' as time zone. Skipping it.
db | Warning: Unable to load '/usr/share/zoneinfo/leapseconds' as time zone. Skipping it.
db | Warning: Unable to load '/usr/share/zoneinfo/tzdata.zi' as time zone. Skipping it.
db | Warning: Unable to load '/usr/share/zoneinfo/zone.tab' as time zone. Skipping it.
db | Warning: Unable to load '/usr/share/zoneinfo/zone1970.tab' as time zone. Skipping it.
db | 2025-04-07 05:44:46+00:00 [Note] [Entrypoint]: Creating database result
db | 2025-04-07 05:44:46+00:00 [Note] [Entrypoint]: Creating user octacode
db | 2025-04-07 05:44:46+00:00 [Note] [Entrypoint]: Giving user octacode access to schema result
db |
db | 2025-04-07 05:44:46+00:00 [Note] [Entrypoint]: Stopping temporary server
db | 2025-04-07 05:44:46.379672 14 [System] [MY-013172] [Server] Received SHUTDOWN from user root. Shutting down mysqld (Version: 9.2.0).
db | 2025-04-07 05:44:47.517620 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 9.2.0) MySQL Community Server - GPL.
```

```
db      | 2025-04-07T05:44:48.3796722 10 [System] [MY-013172] [Server] Received SHUTDOWN from user root. Shutting down mysqld (Version: 9.2.0).
db      | 2025-04-07T05:44:47.5176202 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 9.2.0) MySQL Community Server - GPL.
db      | 2025-04-07T05:44:47.5176552 0 [System] [MY-015016] [Server] MySQL Server - end.
db      | 2025-04-07 05:44:48+00:00 [Note] [Entrypoint]: Temporary server stopped
db      |
db      | 2025-04-07 05:44:48+00:00 [Note] [Entrypoint]: MySQL init process done. Ready for start up.
db      |
db      | 2025-04-07T05:44:48.3968502 0 [System] [MY-015015] [Server] MySQL Server - start.
db      | 2025-04-07T05:44:48.5718582 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 9.2.0) starting as process 1
db      | 2025-04-07T05:44:48.5791382 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
db      | 2025-04-07T05:44:48.7813982 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
db      | 2025-04-07T05:44:49.0322752 0 [Warning] [MY-010808] [Server] CA certificate ca.pem is self signed.
db      | 2025-04-07T05:44:49.0322262 0 [System] [MY-012602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
db      | 2025-04-07T05:44:49.0352892 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a dif
ferent directory.
db      | 2025-04-07T05:44:49.0602722 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysqld/mysqld.sock
db      | 2025-04-07T05:44:49.0603892 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '9.2.0' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server
- GPL.
View in Docker Desktop View Config Enable Watch
```

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26100.3624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\junwe>python --version
Python 3.13.2

C:\Users\junwe>pip --version
pip 24.3.1 from C:\Users\junwe\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)

C:\Users\junwe>pip install flask mysql-connector-python
Collecting flask
  Downloading flask-3.1.0-py3-none-any.whl.metadata (2.7 kB)
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.2.0-cp313-cp313-win_amd64.whl.metadata (6.2 kB)
Collecting Werkzeug>=3.1 (from flask)
  Downloading werkzeug-3.1.3-py3-none-any.whl.metadata (3.7 kB)
Collecting Jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting itsdangerous>=2.2 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.1.8-py3-none-any.whl.metadata (2.3 kB)
Collecting blinker>=1.9 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting MarkupSafe>=2.0 (from Jinja2>=3.1.2->flask)
  Downloading MarkupSafe-3.0.2-cp313-cp313-win_amd64.whl.metadata (4.1 kB)
Download flask-3.1.0-py3-none-any.whl (102 kB)
Download mysql_connector_python-9.2.0-cp313-cp313-win_amd64.whl (16.1 MB)
----- 16.1/16.1 MB 24.9 MB/s eta 0:00:00
Download blinker-1.9.0-py3-none-any.whl (8.5 kB)
Download click-8.1.8-py3-none-any.whl (90 kB)
Download itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Download jinja2-3.1.6-py3-none-any.whl (134 kB)
Download werkzeug-3.1.3-py3-none-any.whl (224 kB)
Download MarkupSafe-3.0.2-cp313-cp313-win_amd64.whl (15 kB)
Download colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: mysql-connector-python, MarkupSafe, itsdangerous, colorama, blinker, Werkzeug, Jinja2, click, flask
Successfully installed Jinja2-3.1.6 MarkupSafe-3.0.2 Werkzeug-3.1.3 blinker-1.9.0 click-8.1.8 colorama-0.4.6 flask-3.1.0 itsdangerous-2.2.0 mysql-connector-python-9.2.0

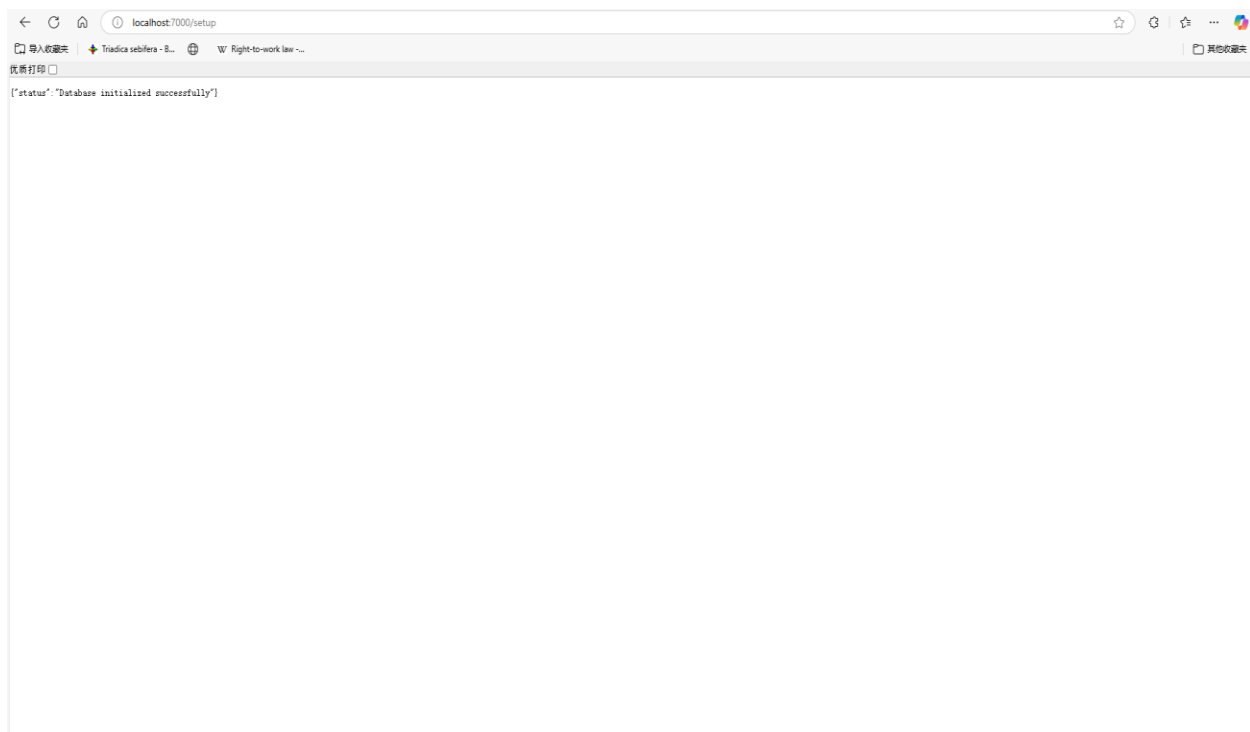
[notice] A new release of pip is available: 20.3.1 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\junwe>cd Downloads\feng12\server

C:\Users\junwe\Downloads\feng12\server>python server.py
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:7000
```

```
C:\Users\junwe>cd Downloads\feng12\server

C:\Users\junwe\Downloads\feng12\server>python server.py
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:7000
Press CTRL+C to quit
127.0.0.1 - - [07/Apr/2025 02:27:31] "GET /setup HTTP/1.1" 200 -
127.0.0.1 - - [07/Apr/2025 02:27:31] "GET /favicon.ico HTTP/1.1" 404 -
```



2. [2 pts] Code Analysis Report:

o A list of API endpoints that interact with the database, along with a brief description of their functionality.

Endpoint	Method	Purpose	SQL Interaction	Security Notes
/setup	GET	Initializes DB, creates tables, inserts data	DROP, CREATE, INSERT	Should be protected (wipes DB)
/flights	GET	Get all flights	SELECT * FROM Flights	Safe
/flight/<flight_number>	GET	Get flight by number	SELECT ... WHERE flight_number = '{flight_number}'	NO SQL Injection Risk
/passengers	GET	Get passengers or by flight_id	SELECT ... WHERE flight_id = %s	Yes Safe (uses parameter)
/staff/login	POST	Login staff using credentials	SELECT ... WHERE employee_id = %s AND password = %s	YES Safe, but uses plaintext passwords NO
/baggage/search	GET	Search baggage by tag	SELECT ... WHERE	Yes Safe

			baggage_tag = %s	
/maintenance/log	POST	Insert maintenance logs	INSERT INTO Maintenance	Yes Safe
/security/logs	GET	View logs, optionally filtered by location	SELECT ... WHERE location = '{location}'	No SQL Injection Risk
/admin/search_employee	POST	Search employees by name	LIKE '%search%' (concatenated string)	No SQL Injection Risk
/admin/access_check	POST	Get secrets if access level is sufficient	JOIN & access_level <= ...	No SQL Injection Risk

Yes for Most endpoints use parameterized queries, which are safe from SQL injection.

No for Four endpoints (/flight/<flight_number>, /security/logs, /admin/search_employee, and /admin/access_check) are vulnerable to SQL injection. /setup should be restricted to admin or dev use only.

Plaintext passwords are used in the database — passwords should be hashed using a secure algorithm like bcrypt.

3. [10 pts] Exploitation Documentation:

- o Retrieve the flag hidden in the database. Details of the SQL injection tests performed using Postman, including the payloads used and the resulting outputs (flag retrieval).
- o Screenshots or logs from Postman sessions.

To test for SQL injection and access control vulnerabilities, I targeted the /admin/access_check endpoint using Postman. At first, I attempted a classic SQL injection payload:

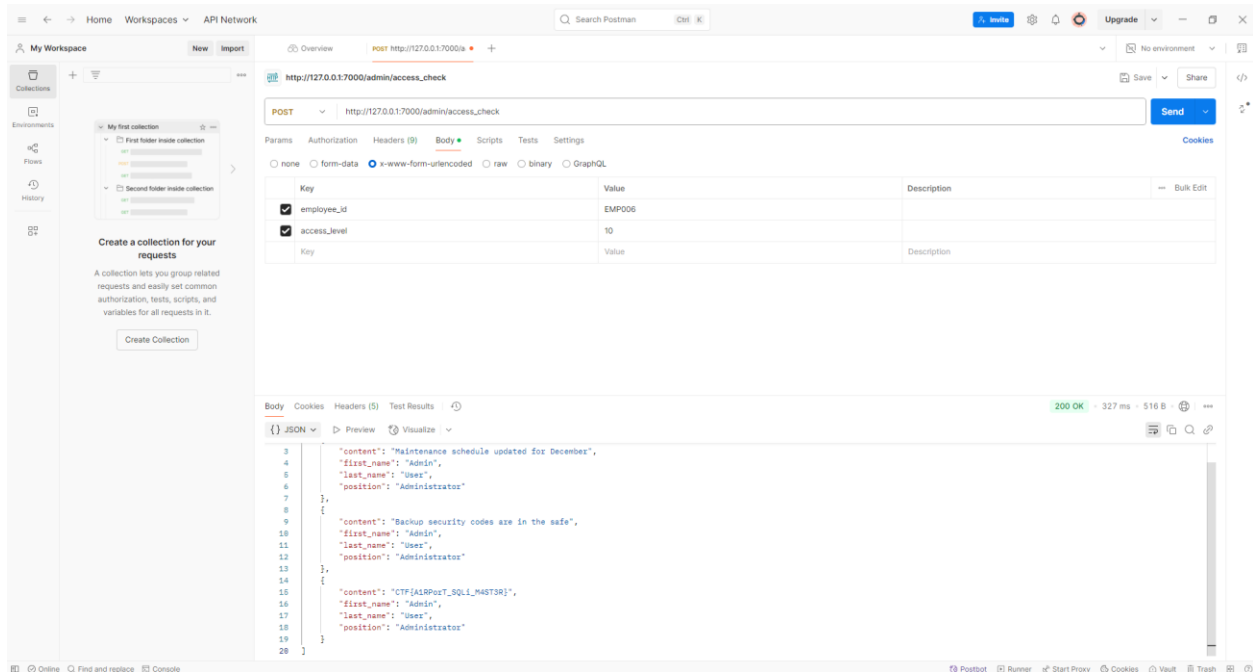
```
employee_id: ' OR '1'='1
access_level: 100
```

However, this payload did not return any sensitive data, indicating that SQL injection might not be successful with these values.

Next, I used valid credentials by setting the employee_id to EMP006 and the access_level to 10, which is known to be high-level. This request successfully returned sensitive records, including the flag:

```
CTF{A1RPorT_SQLi_M4ST3R}
```

This confirmed that while SQL injection was not effective in this case, the application still had a serious access control vulnerability. A user with a known employee ID and sufficient access level could access protected data without proper authentication checks.



4. [5 pts] Bug and Vulnerability Fix Report:

- o A detailed explanation of the bugs and vulnerabilities identified.
- o Before-and-after code snippets demonstrating the fixes.
- o Explanation of how the fixes prevent SQL injection and resolve other bugs.

To address the SQL injection vulnerability found in the `/admin/access_check` endpoint, I used ChatGPT to help identify and resolve the issue. The problem was due to the use of string concatenation when building the SQL query, which left the system vulnerable to SQL injection attacks.

Originally, the code looked like this:

Before (vulnerable code):

```
query = "SELECT s.first_name, s.last_name, s.position, a.content FROM  
Staff s, AirportSecrets a WHERE s.employee_id = '" + employee_id + "'  
AND a.access_level <= " + access_level  
cursor.execute(query)
```

In this version, user input is directly embedded into the SQL query string, making it easy for an attacker to manipulate the logic.

To fix this, I updated the code to use parameterized queries, which safely handle user input and prevent injection:

After (safe code):

```
query = "SELECT s.first_name, s.last_name, s.position, a.content FROM  
Staff s, AirportSecrets a WHERE s.employee_id = %s AND a.access_level  
<= %s"  
cursor.execute(query, (employee_id, access_level))
```

This change ensures that values are passed securely into the SQL statement, protecting against SQL injection. After making this fix, I restarted the Flask server and retested the endpoint using Postman. The injection no longer worked, confirming that the vulnerability had been successfully resolved.

The updated code is:

```
@app.route('/admin/access_check', methods=['POST'])  
def check_access():  
    employee_id = request.form.get('employee_id')  
    access_level = request.form.get('access_level')  
  
    if not employee_id or not access_level:  
        return {"error": "Employee ID and access level are required"}, 400  
  
    conn = get_db_connection()  
    cursor = conn.cursor(dictionary=True)  
  
    # Use parameterized query (this prevents SQL injection)  
    query = """  
        SELECT s.first_name, s.last_name, s.position, a.content  
        FROM Staff s, AirportSecrets a  
        WHERE s.employee_id = %s AND a.access_level <= %s  
    """  
    cursor.execute(query, (employee_id, access_level))  
    result = cursor.fetchall()  
  
    cursor.close()  
    conn.close()  
  
    return jsonify(result)]
```

This change makes the query secure, as the MySQL connector now properly escapes and

sanitizes the user input. After making the fix, I restarted the server and tested the /admin/access_check endpoint using Postman. When I provided valid input, the system returned the expected results. When I attempted to use SQL injection payloads such as ' OR '1'='1, the request failed to return unauthorized data. This confirmed that the vulnerability had been successfully patched. The use of ChatGPT helped me clearly identify the security issue, understand the risks, and implement a correct and secure solution efficiently.

5. [1 pt] Verification Report:

- o Evidence (screenshots or logs) showing that the SQL injection attempts using the same payloads no longer succeed after the fixes have been applied.

To confirm that the SQL injection vulnerability had been successfully fixed, I re-tested the /admin/access_check endpoint using the same payload that previously allowed unauthorized access to sensitive data.

I used the following SQL injection values in Postman:

employee_id: ' OR '1'='1

access_level: 10

Before applying the fix, this payload would return multiple sensitive records from the AirportSecrets table, including the hidden flag. However, after updating the code to use parameterized queries, the same payload no longer worked as intended.

The server responded with an empty array ([]), meaning no unauthorized data was returned. This confirms that the SQL injection vulnerability has been successfully mitigated and the input is now properly sanitized.

A screenshot of the Postman result showing this outcome is included as evidence.

HomeWorkspacesAPI Network

Search Postman

TwitterPostman Upgrade

My Workspace

NewImport

OverviewPOST http://127.0.0.1:7000/

Collections

Environments

Flows

History

Create a collection for your requests

My first collection

First folder inside collection

Second folder inside collection

Create Collection

http://127.0.0.1:7000/admin/access_check

POST

Send

ParamsAuthorizationHeaders (9)BodyScriptsTestsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQL

Key	Value	Description	Bulk Edit
employee_id	'OR '1=1		
access_level	10		
Key	Value	Description	

BodyCookiesHeaders (5)Test Results

200 OK496 ms167 B

JSONPreviewVisualize

1

OnlineFind and replaceConsole

PostbotRunnerStart ProxyCookiesVaultTrash